Documentation

What we were asked

Part 1: propose how to cut a nutritionist's ~25% no-show rate (she runs on WhatsApp and Excel). Part 2: build a small appointments product with a mobile-first client app and a nutritionist dashboard sharing one dataset: book/see/reschedule/cancel on one side; list, status (Booked/Done/No-show, colour coded), add by hand and a Today filter on the other; load five sample appointments; include at least one no-show helper; add anything obviously missing.

Product approach and architecture

Two interfaces, one FastAPI backend, one Supabase PostgreSQL database. React + Vite + TypeScript + Tailwind on Vercel; FastAPI on Render. The client app is built like a consumer wellness app (calm palette, big date numeral for the next appointment, simple day and time picker); the dashboard is a dense, readable table with summary figures. The browser never talks to the database; it only calls the API, and no database secret is in the frontend.

Decisions and assumptions

- **Double booking** is prevented by PostgreSQL itself: a partial unique index on start time for non-cancelled appointments, with the API returning 409. Verified with a 20-way concurrent test.
- **Cancel is a flag, not a status**, so statuses stay exactly Booked / Done / No-show; cancelled rows free their slot.
- **Time:** stored as UTC instants; the API speaks clinic-local date (YYYY-MM-DD) and time (HH:MM) in `Asia/Kolkata`, so the viewer's timezone never matters. "Today" is computed on the server in clinic time.
- **No client accounts:** clients are identified by phone number remembered in the browser. This is not secure authentication (anyone with the number can manage those bookings); documented in the README. Optional `ADMIN_PASSCODE` can protect the dashboard.
- **Assumed practice hours:** 30-minute slots, 09:00–17:00, Mon–Sat, bookable up to 30 days ahead with 1 hour minimum notice. All configurable.
- **Sample data normalisation:** every row stored as `timestamptz` interpreted in Asia/Kolkata; phones reduced to 10 digits ("98200 11111" and "9820011111" are the same number); "11:30am" → 11:30, "4 pm" → 16:00. Meera Iyer's missing phone is stored as NULL, not invented. Dates were not changed; they were not shifted to look upcoming. The seed is re-runnable without duplicates. Note the 25/10/2026 row falls on a Sunday and is kept.

What I added (beyond the brief), and why

- Client "Confirm I'll be there" button + dashboard Confirmed/Unconfirmed markers: turns silent no-shows into known gaps early.
- Calendar (.ics) download with alerts at 1 day and 2 hours before: reminders that need no messaging credentials.
- One-click WhatsApp reminder message and "Reminded" log on the dashboard (sent by the nutritionist).
- "Record outcome" flag for past visits still marked Booked, and a live no-show rate: keeps the data honest.
- Optional dashboard passcode, cancel-by-nutritionist, move-appointment, search and status filters.

What the AI got wrong (and how it was caught and fixed)

Observed during the build (full detail in `docs/ai-issues-log.md`):

1. Search filter used a NUL character as a placeholder, which PostgreSQL rejects (500 error). Caught by an automated test; query rewritten.
2. A malformed appointment id reached the database and caused a 500. Caught by a test; ids are now typed as UUID so the API returns a clean 422, with a regression test.
3. A test wrongly assumed 25 Oct 2026 was a bookable day; it is a Sunday and the practice is closed. The test was wrong, so it was fixed.
4. A global heading colour rule overrode Tailwind classes, making text invisible on the dark "next appointment" card. Caught by looking at a real browser screenshot; styles moved into a CSS layer.
5. A TypeScript construct failed the project's strict compiler setting; caught by the type-check and rewritten.

[Placeholder for you: add anything you find while testing manually, how you noticed it, and how it was fixed.]

How to run it

See `README.md` for full steps. Short version: create a Supabase project, put its connection string in `.env`, run `python database/seed.py`, start the API with `uvicorn app.main:app --reload`, start the web app with `npm run dev`. Tests: `python -m pytest -q` in `backend/` (needs a throwaway Postgres), `npm test && npm run build` in `frontend/`.

Current limitations and remaining setup

Not yet done by me: creating the Supabase project, deploying to Render and Vercel, setting env vars, and checking the hosted app. Automatic WhatsApp/SMS reminders need a provider and credentials. Client identification by phone is not secure authentication. See README "Known limitations".